
Documentação Técnica

Apache Kafka

Infraestrutura Unificada para Processamento

de Dados em Média e Larga Escala

Análise de Dados | Data Mining | Machine Learning

Professor Benir Falcão Pistille

Versão 1.0

Marco 2026



Sumario

Clique com o botão direito no sumario e selecione "Atualizar Campo" para atualizar os números de pagina

Sumario.....	2
1. Introducao	5
1.1 Objetivos.....	5
1.2 Escopo	5
2. Arquitetura Apache Kafka	6
2.1 Componentes Principais	6
2.1.1 Producers (Produtores).....	6
2.1.2 Brokers	6
2.1.3 Topics (Topicos)	7
2.1.4 Consumers (Consumidores).....	7
2.1.5 ZooKeeper / KRaft.....	7
2.2 Topologia e Particionamento	7
3. Instalacao e Configuracao.....	9
3.1 Preparacao do Ambiente.....	9
3.1.1 Requisitos de Hardware	9
3.1.2 Requisitos de Software	9
3.2 Instalacao Passo a Passo.....	10
3.2.1 Download e Extracao	10
3.2.2 Configuracao do KRaft (Sem ZooKeeper).....	10
3.2.3 Configuracao do Servidor.....	10
3.3 Configuracao do Cluster Multi-Broker.....	10
3.3.1 Iniciando o Cluster	10
3.3.2 Testando a Instalacao	11
4. Camadas de Processamento	12
4.1 Camada ETL	12
4.1.1 Kafka Connect	13
4.1.2 Kafka Streams	13

4.1.3 ksqlDB.....	13
4.2 Camada Machine Learning	14
4.2.1 Feature Store	14
4.2.2 MLflow	14
4.2.3 TensorFlow Serving.....	14
4.3 Camada Analise de Dados	14
4.3.1 Apache Superset	15
4.3.2 Grafana.....	15
4.3.3 Jupyter Hub	15
5. Tecnologias Emergentes	16
5.1 Streaming Alternativos.....	16
5.1.1 Redpanda	16
5.1.2 Apache Pulsar.....	16
5.2 Armazenamento e Formatos.....	16
5.2.1 Tiered Storage.....	16
5.2.2 Apache Iceberg.....	17
5.3 Processamento de Streams	17
5.3.1 Materialize	17
5.3.2 RisingWave.....	17
5.4 Observabilidade.....	17
5.4.1 Cruise Control	17
5.4.2 OpenTelemetry	17
6. Referencias e Bibliotecas	18
6.1 Documentacao Oficial	18
6.2 Bibliotecas Cliente	18
6.3 Ferramentas de Integracao	18
6.4 Frameworks de Processamento	18
6.5 Monitoramento e Operacao.....	19
6.6 Machine Learning e Analytics.....	19
6.7 Livros Recomendados.....	19
6.8 Comunidades e Recursos	19

1. Introdução

Apache Kafka é uma plataforma de streaming distribuída de código aberto, desenvolvida pela Apache Software Foundation. Originalmente criada pelo LinkedIn e posteriormente doada para a Apache em 2011, Kafka tornou-se o padrão de fato para processamento de streams de dados em tempo real em escala empresarial.

Este documento apresenta uma Prova de Conceito (POC) completa para instalação e configuração da plataforma Apache Kafka, focada em cenários de processamento de dados em média e larga escala, incluindo análise de dados, data mining (mineração de dados) e aprendizado de máquina.

1.1 Objetivos da documentação

- Demonstrar a instalação completa do Apache Kafka em ambiente de produção
- Configurar um cluster Kafka multi-broker com alta disponibilidade
- Integrar camadas de ETL, Machine Learning e Análise de Dados
- Apresentar tecnologias emergentes no ecossistema Kafka
- Fornecer referências e bibliotecas para implementação

1.1 Objetivos do projeto entregas

- Introdução sobre a solução e Benefícios
- Objetivo do Projeto
- Descrição da plataforma
- BPV (Business Process View) automação de negócios e diagrama
- Fluxograma lógico de Instalação e Configuração
- Referências e Pesquisa

1.2 Escopo

Esta POC (Prova de conceito) abrange desde a instalação básica até a configuração avançada de um cluster Kafka, incluindo integração com ferramentas de processamento de streams, armazenamento de dados e frameworks de machine learning. O documento está organizado em camadas funcionais que refletem uma arquitetura moderna de processamento de dados.

2. Arquitetura Apache Kafka

A arquitetura do Apache Kafka é projetada para alta throughput, baixa latência e escalabilidade horizontal. O sistema é baseado em um modelo de publicação-assinatura (pub/sub) distribuído, onde produtores publicam mensagens em tópicos e consumidores assinam esses tópicos para receber as mensagens.

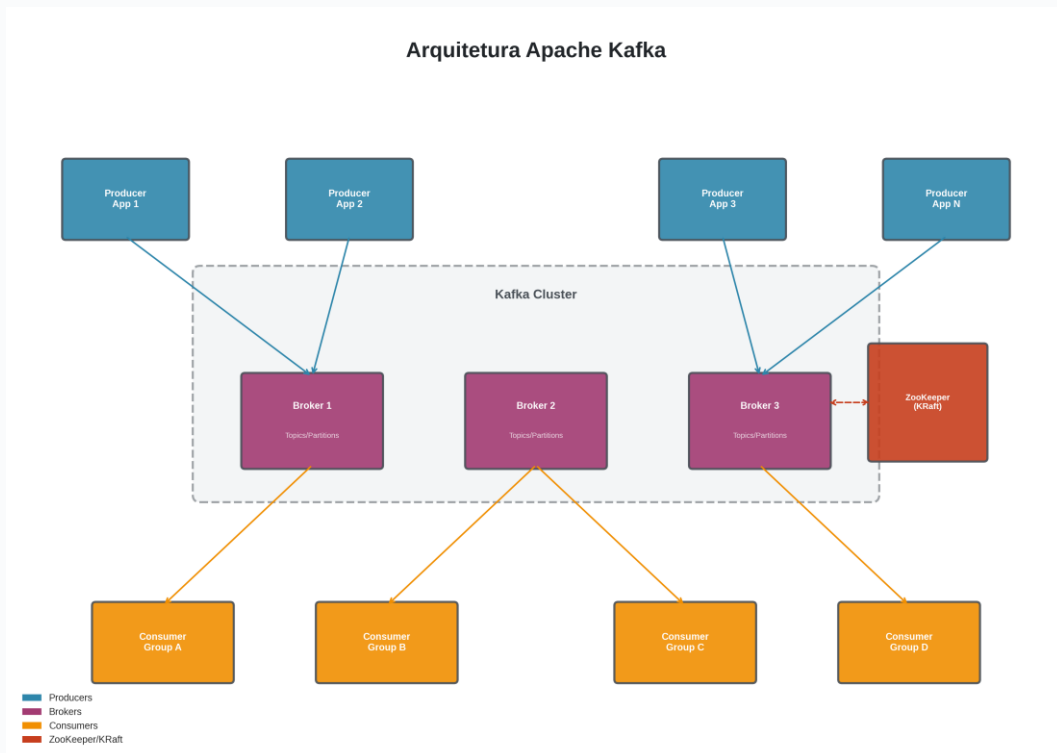


Figura 1: Arquitetura Geral do Apache Kafka

2.1 Componentes Principais

2.1.1 Producers (Produtores)

Produtores são aplicações clientes que publicam (escrevem) eventos em tópicos Kafka. Eles são responsáveis por escolher em qual partição de um tópico o evento será escrito. O Kafka fornece balanceamento de carga automático entre partições, mas também permite que produtores especifiquem chaves para garantir ordenação de mensagens relacionadas.

2.1.2 Brokers

Brokers são servidores Kafka que armazenam e servem dados. Um cluster Kafka consiste em múltiplos brokers para garantir alta disponibilidade e escalabilidade. Cada broker é identificado por um ID único e pode gerenciar múltiplas partições de diferentes tópicos.

2.1.3 Topics (Tópicos)

Tópicos são categorias ou feeds de nomeação para os quais os produtores publicam mensagens. Cada tópico é dividido em partições, que são unidades de paralelismo e replicação. As partições são distribuídas entre os brokers do cluster.

2.1.4 Consumers (Consumidores)

Consumidores leem dados de tópicos Kafka. Eles podem ser organizados em grupos de consumidores (consumer groups), onde cada partição é consumida por apenas um consumidor do grupo, permitindo processamento paralelo e balanceamento de carga.

2.1.5 ZooKeeper / KRaft

Tradicionalmente, o Kafka dependia do Apache ZooKeeper para coordenação de cluster, gerenciamento de configurações e eleição de líderes. A partir da versão 3.0, o Kafka introduziu o modo KRaft (Kafka Raft), que elimina a dependência do ZooKeeper, simplificando a arquitetura e melhorando a performance.

2.2 Topologia e Particionamento

O particionamento é um conceito fundamental no Kafka que permite a escalabilidade horizontal. Cada tópico é dividido em um número configurável de partições, e cada partição é um log ordenado e imutável de mensagens.

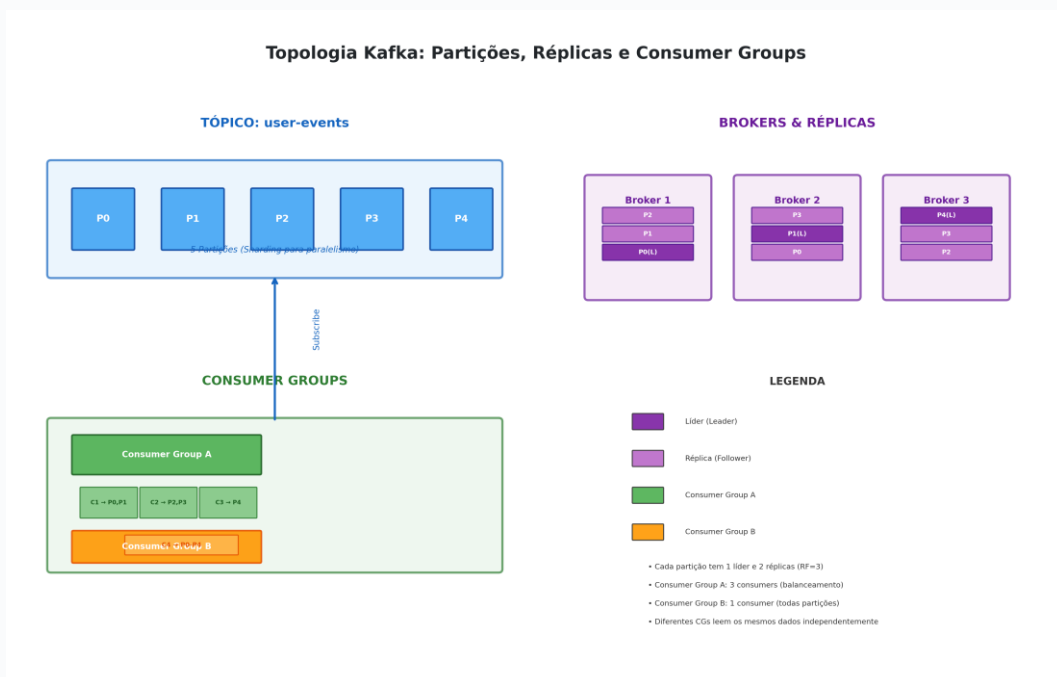


Figura 2: Topologia Kafka - Partições, Réplicas e Consumer Groups

A replicação de partições garante alta disponibilidade. Cada partição tem um líder (leader) e zero ou mais seguidores (followers). O líder processa todas as leituras e escritas, enquanto os seguidores replicam os dados para failover.

3. Instalação e Configuração

Esta seção apresenta o processo completo de instalação e configuração do Apache Kafka, desde a preparação do ambiente até a configuração de um cluster multi-broker em produção.

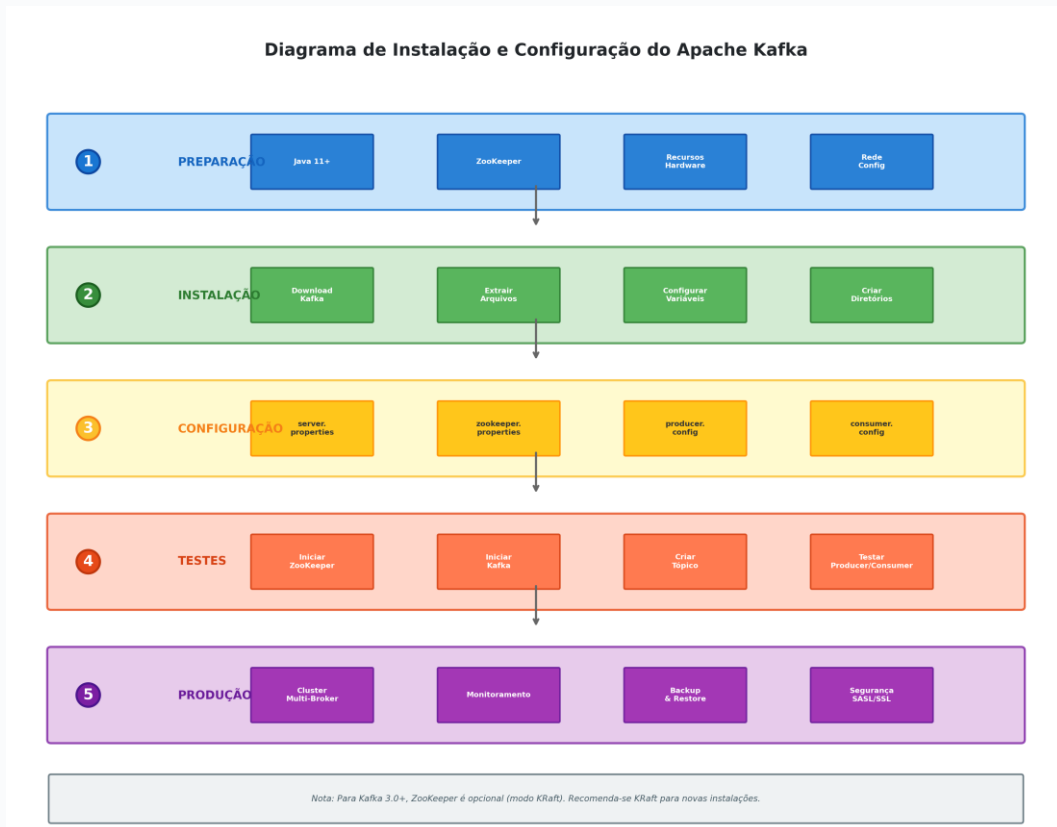


Figura 3: Diagrama de Instalação e Configuração do Apache Kafka

3.1 Preparação do Ambiente

3.1.1 Requisitos de Hardware

- CPU: Mínimo 4 cores, recomendado 8+ cores para produção
- Memória: Mínimo 8GB RAM, recomendado 16-32GB para clusters
- Disco: SSD recomendado, mínimo 100GB livre
- Rede: Conexão estável de 1Gbps ou superior

3.1.2 Requisitos de Software

- Sistema Operacional: Linux (Ubuntu 20.04+, CentOS 7+, RHEL 8+)

- Java: OpenJDK 11 ou superior
- ZooKeeper: 3.5.7+ (apenas para modos legados)

3.2 Instalação Passo a Passo

3.2.1 Download e Extração

```
# Download do Kafka
wget https://downloads.apache.org/kafka/3.6.1/kafka_2.13-3.6.1.tgz
tar -xzf kafka_2.13-3.6.1.tgz
cd kafka_2.13-3.6.1
```

3.2.2 Configuração do KRaft (Sem ZooKeeper)

A partir do Kafka 3.3+, o modo KRaft está pronto para produção. Esta configuração elimina a necessidade do ZooKeeper.

```
# Gerar ID do cluster
KAFKA_CLUSTER_ID="$(bin/kafka-storage.sh random-uuid)"
bin/kafka-storage.sh format -t $KAFKA_CLUSTER_ID -c config/kraft/server.properties
```

3.2.3 Configuração do Servidor

```
# Editar config/kraft/server.properties
node.id=1
controller.quorum.voters=1@localhost:9093
listeners=PLAINTEXT://:9092,CONTROLLER://:9093
log.dirs=/var/lib/kafka/logs
```

3.3 Configuração do Cluster Multi-Broker

Para alta disponibilidade, configure um cluster com múltiplos brokers. Cada broker deve ter um node.id único e compartilhar a configuração de quórum.

```
# Configuracao para 3 brokers
# Broker 1: node.id=1
# Broker 2: node.id=2
# Broker 3: node.id=3
controller.quorum.voters=1@broker1:9093,2@broker2:9093,3@broker3:9093
```

3.3.1 Iniciando o Cluster

```
# Iniciar cada broker
```

```
bin/kafka-server-start.sh config/kraft/server.properties
```

3.3.2 Testando a Instalação

```
# Criar um topico
```

```
bin/kafka-topics.sh --create --topic test-topic --bootstrap-server localhost:9092 -  
-partitions 3 --replication-factor 1
```

```
# Listar topicos
```

```
bin/kafka-topics.sh --list --bootstrap-server localhost:9092
```

4. Camadas de Processamento

A arquitetura de processamento de dados com Kafka é organizada em camadas funcionais que trabalham em conjunto para fornecer um pipeline completo desde a ingestão até a análise e machine learning.

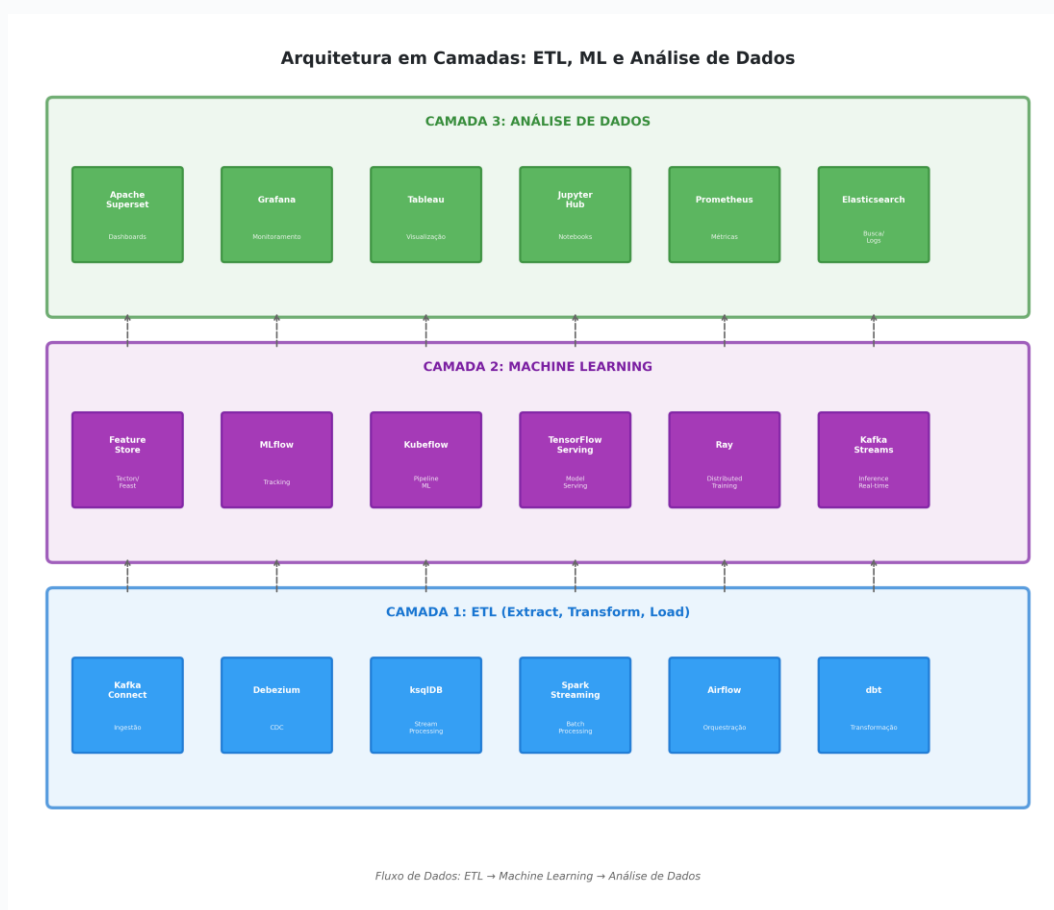


Figura 4: Arquitetura em Camadas - ETL, ML e Análise de Dados

4.1 Camada ETL

A camada ETL (Extract, Transform, Load) é responsável pela ingestão, processamento e transformação de dados. No ecossistema Kafka, esta camada utiliza diversas ferramentas especializadas.

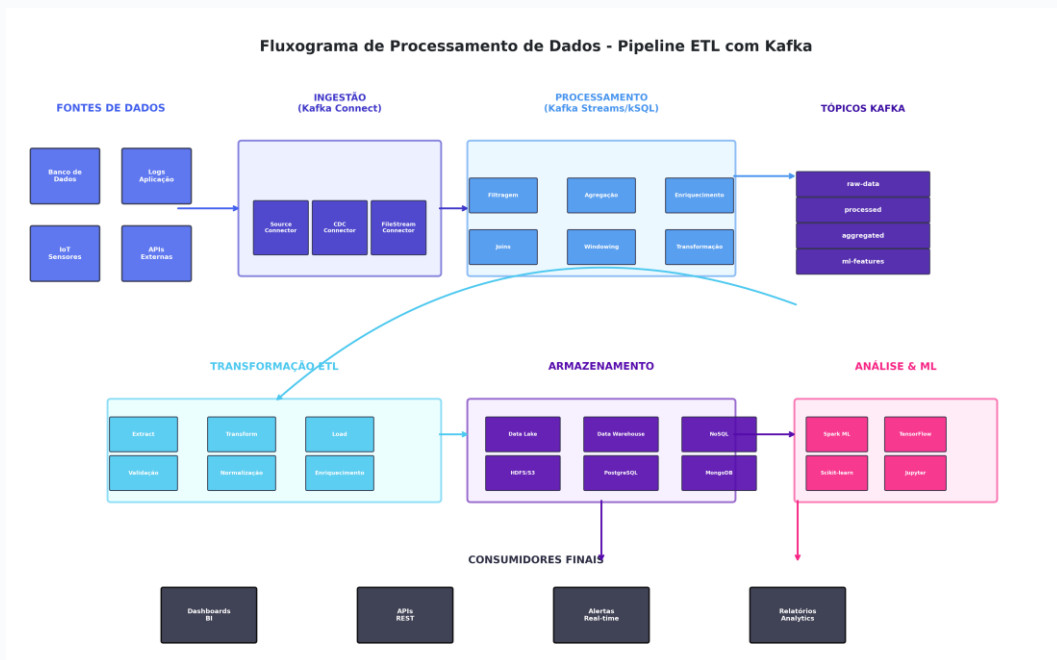


Figura 5: Fluxograma de Processamento de Dados - Pipeline ETL

4.1.1 Kafka Connect

Kafka Connect é um framework para conectar Kafka com sistemas externos de forma escalável e confiável. Ele suporta conectores source (para importar dados) e sink (para exportar dados).

Exemplo: Conector JDBC Source

```
curl -X POST http://localhost:8083/connectors -H "Content-Type: application/json" -d '{
  "name": "jdbc-source-connector",
  "config": {
    "connector.class": "io.confluent.connect.jdbc.JdbcSourceConnector",
    "connection.url": "jdbc:postgresql://localhost:5432/mydb",
    "mode": "incrementing",
    "incrementing.column.name": "id"
  }
}'
```

4.1.2 Kafka Streams

Kafka Streams é uma biblioteca cliente para construir aplicações e micro serviços de processamento de streams, onde os dados de entrada e saída são armazenados em clusters Kafka.

4.1.3 ksqlDB

ksqlDB é um motor de processamento de streams SQL que torna fácil processar dados em tempo real no Kafka usando comandos SQL familiares.

```
-- Criar stream a partir de topico  
CREATE STREAM user_events (  
  user_id VARCHAR,  
  event_type VARCHAR,  
  timestamp BIGINT  
) WITH (KAFKA_TOPIC='user-events', VALUE_FORMAT='JSON');
```

4.2 Camada Machine Learning

A integração de Kafka com frameworks de machine learning permite a construção de pipelines de ML em tempo real, desde a ingestão de dados de treinamento até a implantação de modelos.

4.2.1 Feature Store

Feature Stores como Feast e Tecton permitem armazenar e servir features para treinamento e inferência de modelos de ML, com integração direta ao Kafka para features em tempo real.

4.2.2 MLflow

MLflow é uma plataforma open source para o ciclo de vida de ML, incluindo experimentação, reprodução, implantação e um registro de modelos centralizado.

4.2.3 TensorFlow Serving

TensorFlow Serving é um sistema de serving flexível e de alta performance para modelos de machine learning, projetado para ambientes de produção, com integração via Kafka para inferência em tempo real.

4.3 Camada Análise de Dados

A camada de análise de dados fornece ferramentas para visualização, monitoramento e análise dos dados processados pelo Kafka.

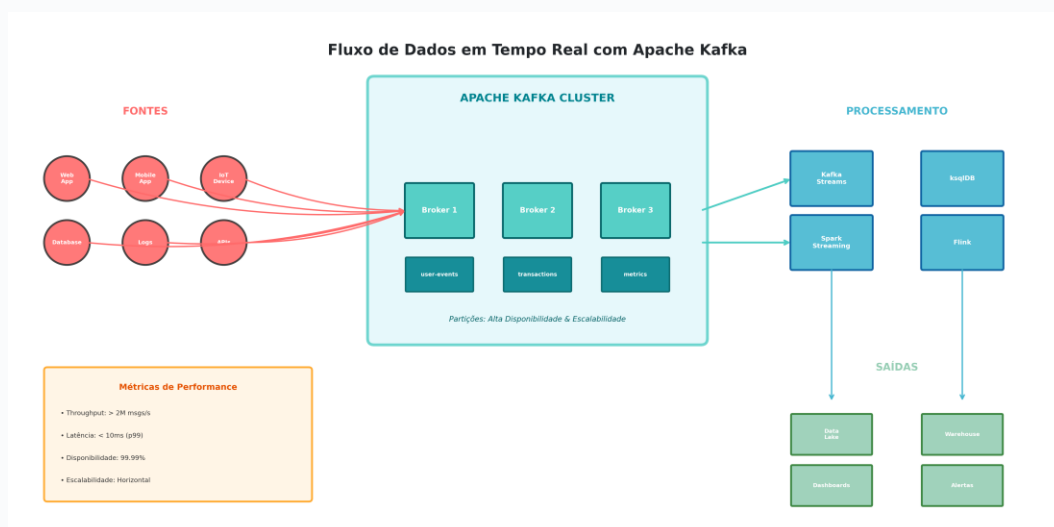


Figura 6: Fluxo de Dados em Tempo Real com Apache Kafka

4.3.1 Apache Superset

Apache Superset é uma aplicação web moderna e intuitiva para exploração e visualização de dados. Ele se integra facilmente com bancos de dados populares e pode consumir dados processados pelo Kafka.

4.3.2 Grafana

Grafana é uma plataforma de análise e monitoramento interativa de código aberto. Com o plugin Kafka, é possível criar dashboards em tempo real para monitoramento de tópicos, lag de consumidores e métricas de performance.

4.3.3 Jupyter Hub

Jupyter Hub fornece notebooks multiusuário para análise exploratória de dados, permitindo que cientistas de dados e analistas explorem dados do Kafka usando Python, Scala ou R.

5. Tecnologias Emergentes

O ecossistema Kafka continua evoluindo com novas tecnologias que expandem suas capacidades e simplificam sua operação. A seguir, apresentamos as principais tendências e tecnologias emergentes.

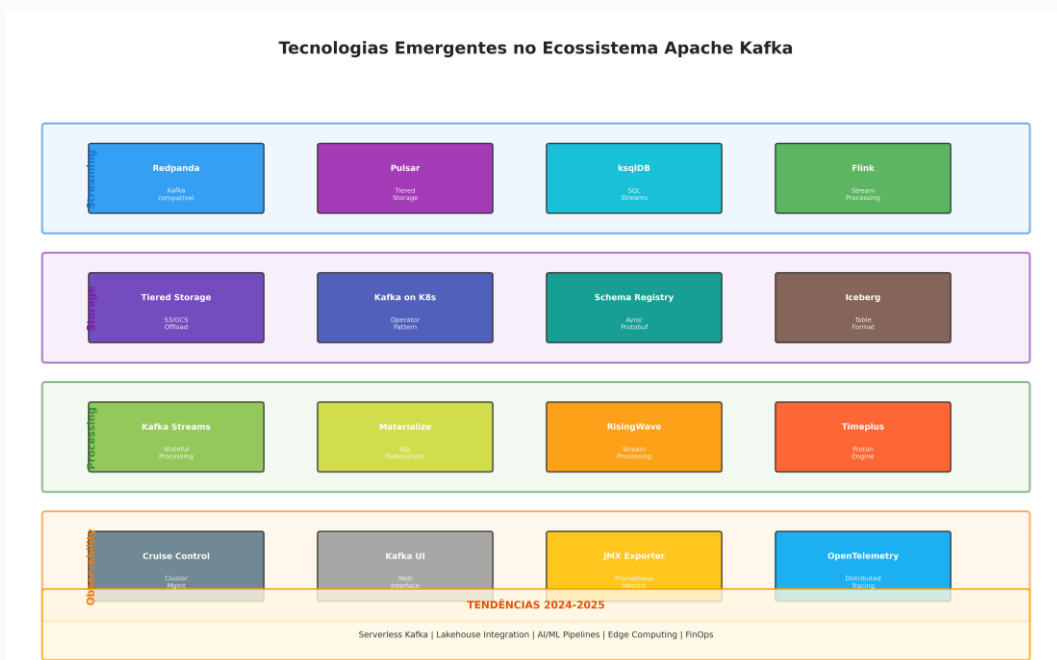


Figura 7: Tecnologias Emergentes no Ecossistema Apache Kafka

5.1 Streaming Alternativos

5.1.1 Redpanda

Redpanda é uma alternativa moderna ao Kafka, compatível com a API Kafka, mas escrita em C++ para melhor performance e sem dependência do ZooKeeper ou JVM.

5.1.2 Apache Pulsar

Apache Pulsar é uma plataforma de streaming distribuída que oferece tiered storage (armazenamento em camadas), multi-tenancy nativo e geo-replicação.

5.2 Armazenamento e Formatos

5.2.1 Tiered Storage

O Tiered Storage permite que dados antigos sejam movidos automaticamente para armazenamento de objeto (S3, GCS) reduzindo custos enquanto mantém a capacidade de leitura.

5.2.2 Apache Iceberg

Apache Iceberg é um formato de tabela open source para grandes conjuntos de dados analíticos, que se integra com Kafka para construção de lakehouses.

5.3 Processamento de Streams

5.3.1 Materialize

Materialize é um motor de processamento de streams SQL que mantém views materializadas atualizadas em tempo real a partir de streams de dados.

5.3.2 RisingWave

RisingWave é um banco de dados de processamento de streams distribuído que permite executar consultas SQL contínuas em dados em movimento.

5.4 Observabilidade

5.4.1 Cruise Control

LinkedIn Cruise Control é um sistema para automatizar a operação de clusters Kafka, incluindo balanceamento de carga, detecção de anomalias e autopreparação.

5.4.2 OpenTelemetry

OpenTelemetry fornece instrumentação padronizada para tracing distribuído e métricas, essencial para observabilidade em arquiteturas baseadas em Kafka.

6. Referencias e Bibliotecas

6.1 Documentação Oficial

0. **Apache Kafka Documentation:** <https://kafka.apache.org/documentation/>
1. **Confluent Documentation:** <https://docs.confluent.io/>
2. **Kafka Improvement Proposals (KIPs):**
<https://cwiki.apache.org/confluence/display/KAFKA/Kafka+Improvement+Proposals>

6.2 Bibliotecas Cliente

- kafka-python: Cliente Python para Apache Kafka
- kafka-clients: Cliente Java oficial
- librdkafka: Cliente C/C++ de alta performance
- confluent-kafka-go: Cliente Go
- kafka-js: Cliente Node.js

6.3 Ferramentas de Integração

- Kafka Connect: Framework de integração nativo
- Debezium: CDC (Change Data Capture) para bancos de dados
- Camel Kafka Connector: Conectores Apache Camel
- StreamSets: Plataforma de ingestão de dados

6.4 Frameworks de Processamento

- Kafka Streams: Processamento de streams nativo
- ksqlDB: SQL para streams Kafka
- Apache Flink: Processamento de streams avançado
- Apache Spark Streaming: Processamento em micro-batches
- Apache Storm: Processamento de eventos em tempo real

6.5 Monitoramento e Operação

- Kafka Manager/Yahoo Kafka Manager: Gerenciamento de clusters
- Kafka UI: Interface web para Kafka
- Cruise Control: Automação de operações
- Prometheus + Grafana: Monitoramento e visualização
- LinkedIn Burrow: Monitoramento de lag de consumidores

6.6 Machine Learning e Analytics

- Feast: Feature Store open source
- MLflow: Plataforma de ciclo de vida de ML
- TensorFlow Extended (TFX): Pipeline de ML em produção
- Apache Superset: Visualização e dashboards
- Jupyter: Notebooks interativos

6.7 Livros Recomendados

3. Kafka: The Definitive Guide - Neha Narkhede, Gwen Shapira, Todd Palino (O'Reilly)
4. Designing Event-Driven Systems - Ben Stopford (O'Reilly)
5. Streaming Systems - Tyler Akidau, Slava Chernyak, Reuven Lax (O'Reilly)
6. Building Data-Intensive Applications - Martin Kleppmann (O'Reilly)

6.8 Comunidades e Recursos

- Apache Kafka Mailing Lists: user@kafka.apache.org, dev@kafka.apache.org
- Confluent Community: <https://community.confluent.io/>
- Stack Overflow: Tag [apache-kafka]
- GitHub: <https://github.com/apache/kafka>
- Meetups: Kafka Meetups globais

Documentação técnica Apache Kafka

Processamento de Dados em Média e Larga Escala

Análise de Dados | Data Mining | Machine Learning

Professor Benir Falcão Pistille

Versão 1.0

Marco 2026